

Sistemas Operativos en Tiempo Real

Codiseño HW/SW – Sistemas Integrados Digitales

Marcos Martínez Peiró mpeiro@eln.upv.es

*Máster Universitario de Ingeniería de Sistemas Electrónicos – MUISE –
Máster Universitario de Ingeniería de Telecomunicaciones – MUIT -*

Rtos –Real Time Operating Systems

Organización

- Sesión 1. Introducción.
 - Concepto de Sistema Embebido.
 - Arquitectura SW de un Sistema Embebido.
 - Concepto de Rtos y Principales Rtos.
- **Sesión 2. Micro C**
 - Inicialización sobre NIOS II.
 - Tareas.
 - Gestión de Tiempo.
 - Interrupciones.
 - Descripción Lab1.
- Sesión 3. Micro C
 - Comunicación entre tareas.
 - Gestión de Memoria.
 - Timers.
 - Hooks.
 - Descripción Lab 2 y Lab 3

Sesión 2

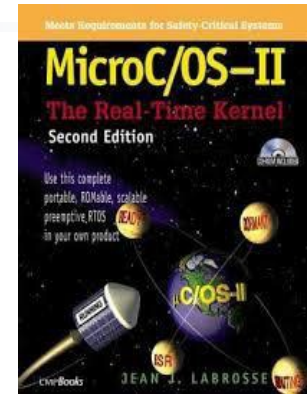
Micro C (UCOS)

- Inicialización sobre NIOS II.
- Tareas.
- Gestión de Tiempo.
- Interrupciones.
- Descripción Lab1.

Micro C

Características

- Sistema operativo de tiempo real desarrollado por J. Labrosse desde 1992. Comercializado por Micrium (www.micrium.com)
- Certificado para requerimientos de aviación y automoción.
- Libro : MicroC/OS-II The Real-Time Kernel, 2nd ed, 2002 (disponible en internet). Versión MicroC/OS-III disponible.
- Adquirida por SiliconLabs
- <https://micrium.atlassian.net/wiki/spaces/ucos/overview>
- [Recursos opensource](#)
- <https://github.com/weston-embedded/uC-OS2>



Micro C

Características

- **Código fuente** disponible y bien documentado (aprox. 20 KB)
- **Portable** (ARM, NIOSII, procesadores de 8/16/32/64 bits, DSP, ...).
- **ROMable**: fácil integración en sistemas empotrados.
- **Escalable**: los servicios deseados pueden incluirse de forma individual mediante compilación condicional (`#define constant`).
- **Expulsivo (preemptive)**: siempre ejecuta la tarea más prioritaria lista.
- **Multitarea** (64 tareas, 256 v>2.8).
- **Determinista**: mismos tiempos de ejecución de la mayoría de servicios.
- **Pilas por tareas**: $\mu\text{C}/\text{OS-II}$ permite que cada tarea tenga un tamaño de pila diferentes. Uso eficiente de la memoria.
- **Interrupciones**: soporta interrupciones anidadas hasta en 255 niveles.

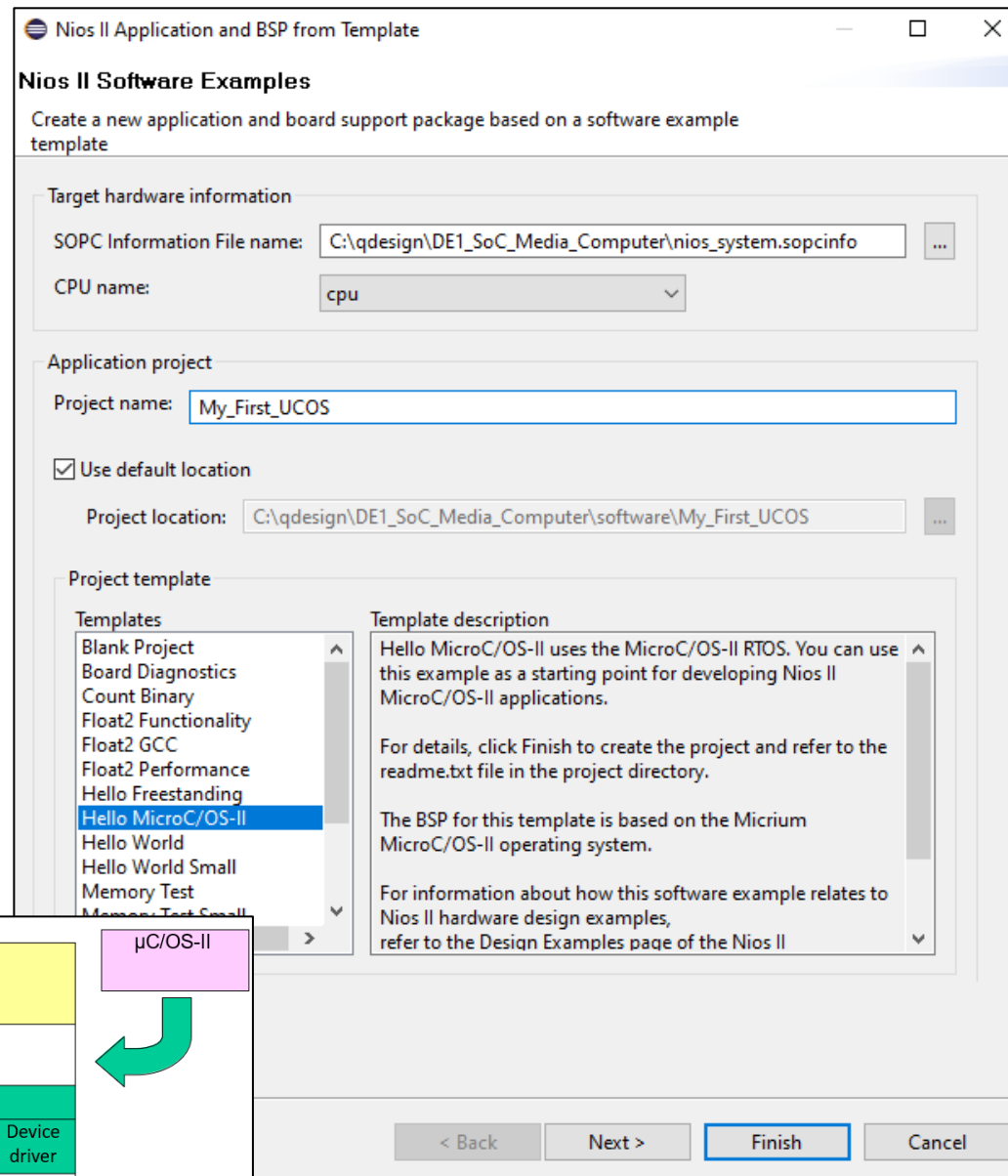
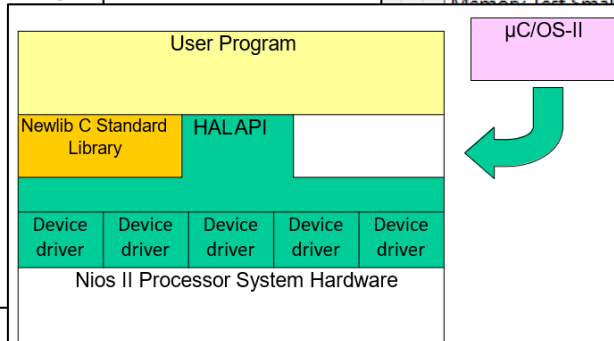
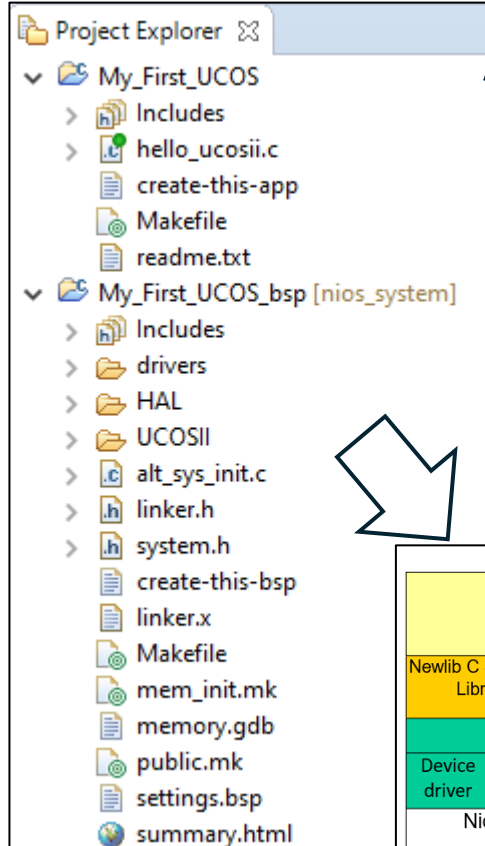
Inicialización

- UCOS está portado sobre el procesador NIOSII.
 - Existen *templates* que permiten crear el sistema de ficheros que soporta el RTOS UCOS.
 - Al seleccionar una *template* desde Eclipse con un ejemplo basado en UCOS se crean las carpetas de la portabilidad.
 - Podremos usar los servicios del RTOS (llamadas a funciones).

Micro C

Inicialización

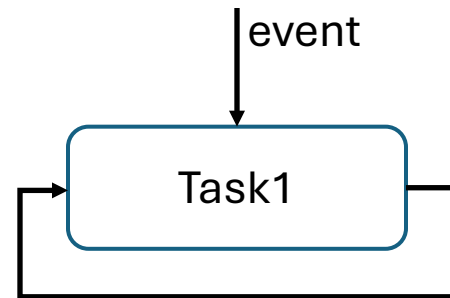
- Templates desde Eclipse – Quartus
- Portabilidad



UCOS-II

Tasks

- Una tarea está definida por:
 - La pila
 - La prioridad
 - La función que realiza (código)
- La tarea suele ser un **bucle infinito** con un **evento** que interrumpe el bucle y permite abandonar la CPU.
- Todo ello debe ser programado por el desarrollador.
- Debemos usar los servicios de UCOS para crear tareas y para gestionar los eventos.



UCOS-II

Tasks

- Ejemplo desde *template*.
- Uso de Servicios para crear tareas. *OSTaskCreateExt()*.
- Inicialización de UCOS: *OSInit()*. No se necesita en Quartus (portado por defecto).
- Arranque de scheduler. *OSStart()*;

```
/* The main function creates two task and starts multi-tasking */  
int main(void)  
{  
    OSInit();  
  
    OSTaskCreateExt(task1,  
                    NULL,  
                    (void *)&task1_stk[TASK_STACKSIZE-1],  
                    TASK1_PRIORITY,  
                    TASK1_PRIORITY,  
                    task1_stk,  
                    TASK_STACKSIZE,  
                    NULL,  
                    0);  
  
    OSTaskCreateExt(task2,  
                    NULL,  
                    (void *)&task2_stk[TASK_STACKSIZE-1],  
                    TASK2_PRIORITY,  
                    TASK2_PRIORITY,  
                    task2_stk,  
                    TASK_STACKSIZE,  
                    NULL,  
                    0);  
  
    OSStart();  
    return 0;  
}
```

- Inicializa variables y estructuras de datos de $\mu\text{C}/\text{OS-II}$.
- Crea la *Idle Task* (*OSTaskIdle()*), prioridad *OS_LOWEST_PRIO* [63].
- Crea *OSTaskStat()* si configurada, prioridad *OS_LOWEST_PRIO-1* [62].

} *OSInit()*

UCOS-II

Tasks

- Los elementos de la tarea deben estar definidos.
- Pila y prioridad.
- Función o programa de la tarea.
- En el mismo .c o en otros ficheros.
- “*includes.h*” incluye las librerías de UCOS.

```
#include <stdio.h>
#include "includes.h"

/* Definition of Task Stacks */
#define TASK_STACKSIZE 2048
OS_STK task1_stk[TASK_STACKSIZE];
OS_STK task2_stk[TASK_STACKSIZE];

/* Definition of Task Priorities */
#define TASK1_PRIORITY 1
#define TASK2_PRIORITY 2

/* Prints "Hello World" and sleeps for three seconds */
void task1(void* pdata)
{
    while (1)
    {
        printf("Hello from task1\n");
        OSTimeDlyHMSM(0, 0, 3, 0);
    }
}

/* Prints "Hello World" and sleeps for three seconds */
void task2(void* pdata)
{
    while (1)
    {
        printf("Hello from task2\n");
        OSTimeDlyHMSM(0, 0, 3, 0);
    }
}
```

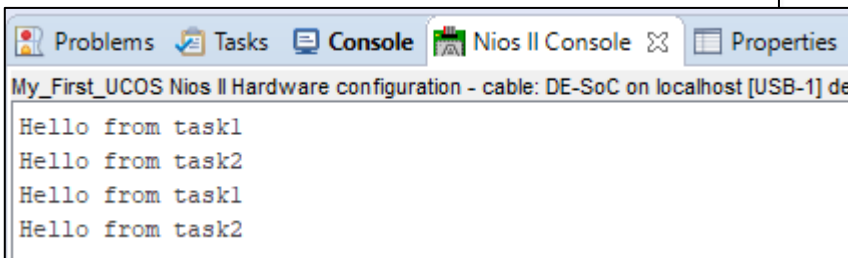
Tamaño de pila

Puntero de pila

Prioridades

Evento

Bucle infinito con evento



The screenshot shows a console window with the following output:

```
Hello from task1
Hello from task2
Hello from task1
Hello from task2
```

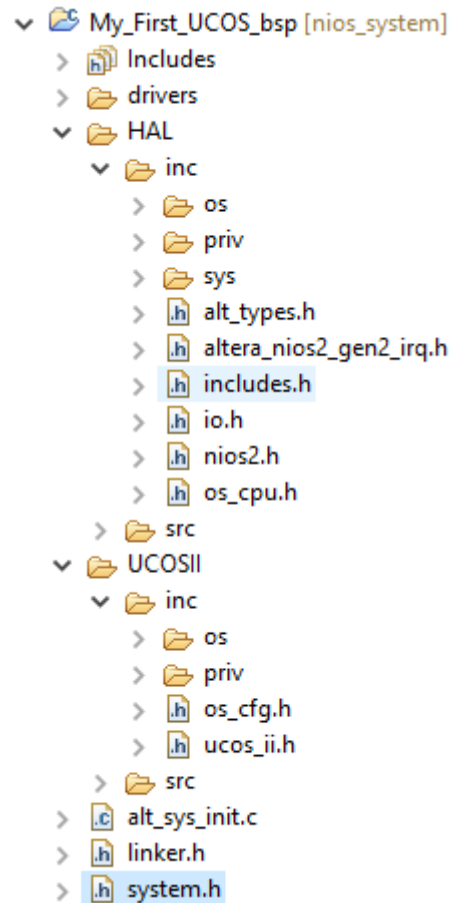


Sistema en ejecución por consola

UCOS-II

Tasks

- Ficheros de interés en el proyecto.
- “*includes.h*” incluye las librerías de UCOS.
- “*os_cpu.h*” configuración de UCOS.
- “*system.h*” ha sido modificado.

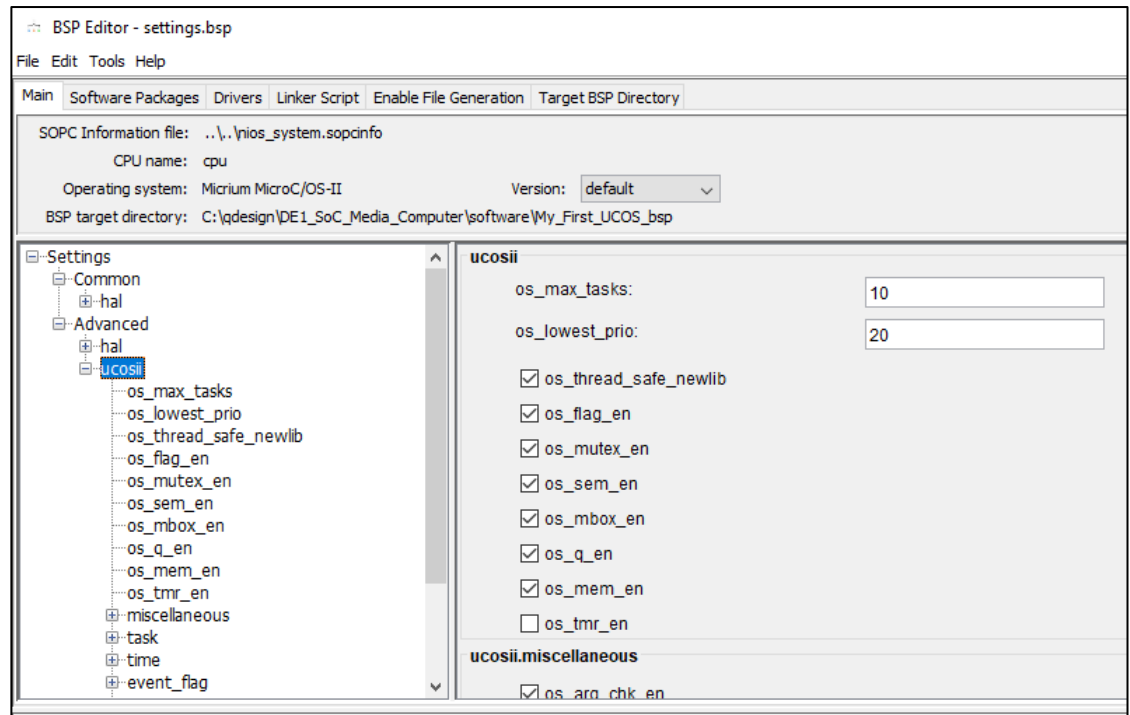
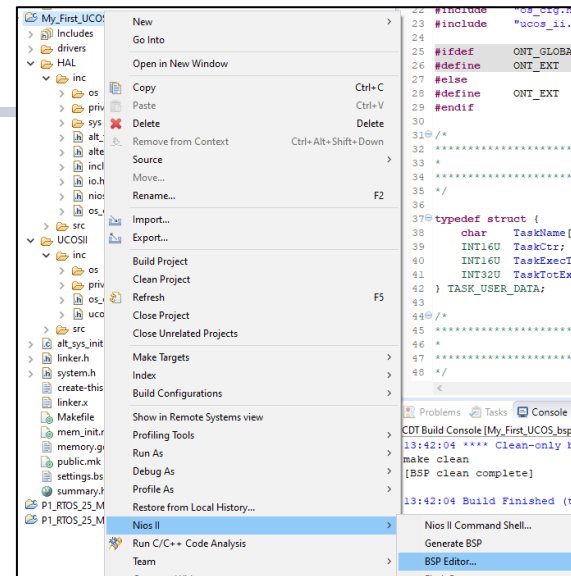


Los ficheros de UCOS se distribuyen entre carpetas HAL y UCOSII. No se suelen editar a mano.

UCOS-II

Tasks - Configuración

- Desde la carpeta BSP: NiosII-> BSP Editor.
- Número de tareas.
- Selección de servicios a usar.
- Adapta tamaño a requerimientos.
- Por defecto casi todo activo.



UCOS-II

Configuración

- “**system.h**” ha sido modificado para UCOS portabilidad.
- 1 -> activo servicio, 0-> no activo servicio. Reflejo de Editor de BSP.
- La clave de funcionamiento es el reloj del sistema. Es la referencia temporal del *Scheduler*.

```
/*
 * ucosii configuration
 *
 */

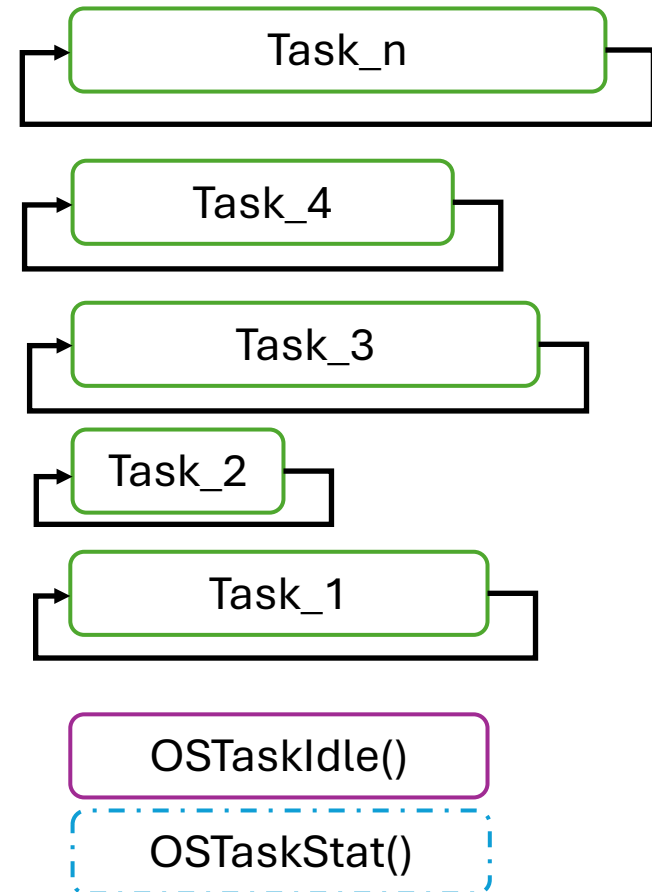
#define OS_ARG_CHK_EN 1
#define OS_CPU_HOOKS_EN 1
#define OS_DEBUG_EN 1
#define OS_EVENT_NAME_SIZE 32
#define OS_FLAGS_NBITS 16
#define OS_FLAG_ACCEPT_EN 1
#define OS_FLAG_DEL_EN 1
#define OS_FLAG_EN 1
#define OS_FLAG_NAME_SIZE 32
#define OS_FLAG_QUERY_EN 1
#define OS_FLAG_WAIT_CLR_EN 1
#define OS_LOWEST_PRIO 20
#define OS_MAX_EVENTS 60
#define OS_MAX_FLAGS 20
#define OS_MAX_MEM_PART 60
#define OS_MAX_QS 20
#define OS_MAX_TASKS 10
#define OS_MBOX_ACCEPT_EN 1
#define OS_MBOX_DEL_EN 1
#define OS_MBOX_EN 1
#define OS_MBOX_POST_EN 1
#define OS_MBOX_POST_OPT_EN 1
#define OS_MBOX_QUERY_EN 1
#define OS_MEM_EN 1
#define OS_MEM_NAME_SIZE 32
#define OS_MEM_QUERY_EN 1
#define OS_MUTEX_ACCEPT_EN 1
#define OS_MUTEX_DEL_EN 1
#define OS_MUTEX_EN 1
#define OS_MUTEX_QUERY_EN 1
#define OS_Q_ACCEPT_EN 1
#define OS_Q_DEL_EN 1

#define OS_Q_DEL_EN 1
#define OS_Q_EN 1
#define OS_Q_FLUSH_EN 1
#define OS_Q_POST_EN 1
#define OS_Q_POST_FRONT_EN 1
#define OS_Q_POST_OPT_EN 1
#define OS_Q_QUERY_EN 1
#define OS_SCHED_LOCK_EN 1
#define OS_SEM_ACCEPT_EN 1
#define OS_SEM_DEL_EN 1
#define OS_SEM_EN 1
#define OS_SEM_QUERY_EN 1
#define OS_SEM_SET_EN 1
#define OS_TASK_CHANGE_PRIO_EN 1
#define OS_TASK_CREATE_EN 1
#define OS_TASK_CREATE_EXT_EN 1
#define OS_TASK_DEL_EN 1
#define OS_TASK_IDLE_STK_SIZE 512
#define OS_TASK_NAME_SIZE 32
#define OS_TASK_PROFILE_EN 1
#define OS_TASK_QUERY_EN 1
#define OS_TASK_STAT_EN 1
#define OS_TASK_STAT_STK_CHK_EN 1
#define OS_TASK_STAT_STK_SIZE 512
#define OS_TASK_SUSPEND_EN 1
#define OS_TASK_SW_HOOK_EN 1
#define OS_TASK_TMR_PRIO 0
#define OS_TASK_TMR_STK_SIZE 512
#define OS_THREAD_SAFE_NEWLIB 1
#define OS_TICKS_PER_SEC TIMER TICKS PER SEC
#define OS_TICK_STEP_EN 1
#define OS_TIME_DLY_HMSM_EN 1
#define OS_TIME_DLY_RESUME_EN 1
#define OS_TIME_GET_SET_EN 1
#define OS_TIME_TICK_HOOK_EN 1
#define OS_TMR_CFG_MAX 16
#define OS_TMR_CFG_NAME_SIZE 16
#define OS_TMR_CFG_TICKS_PER_SEC 10
#define OS_TMR_CFG_WHEEL_SIZE 2
#define OS_TMR_EN 0
```

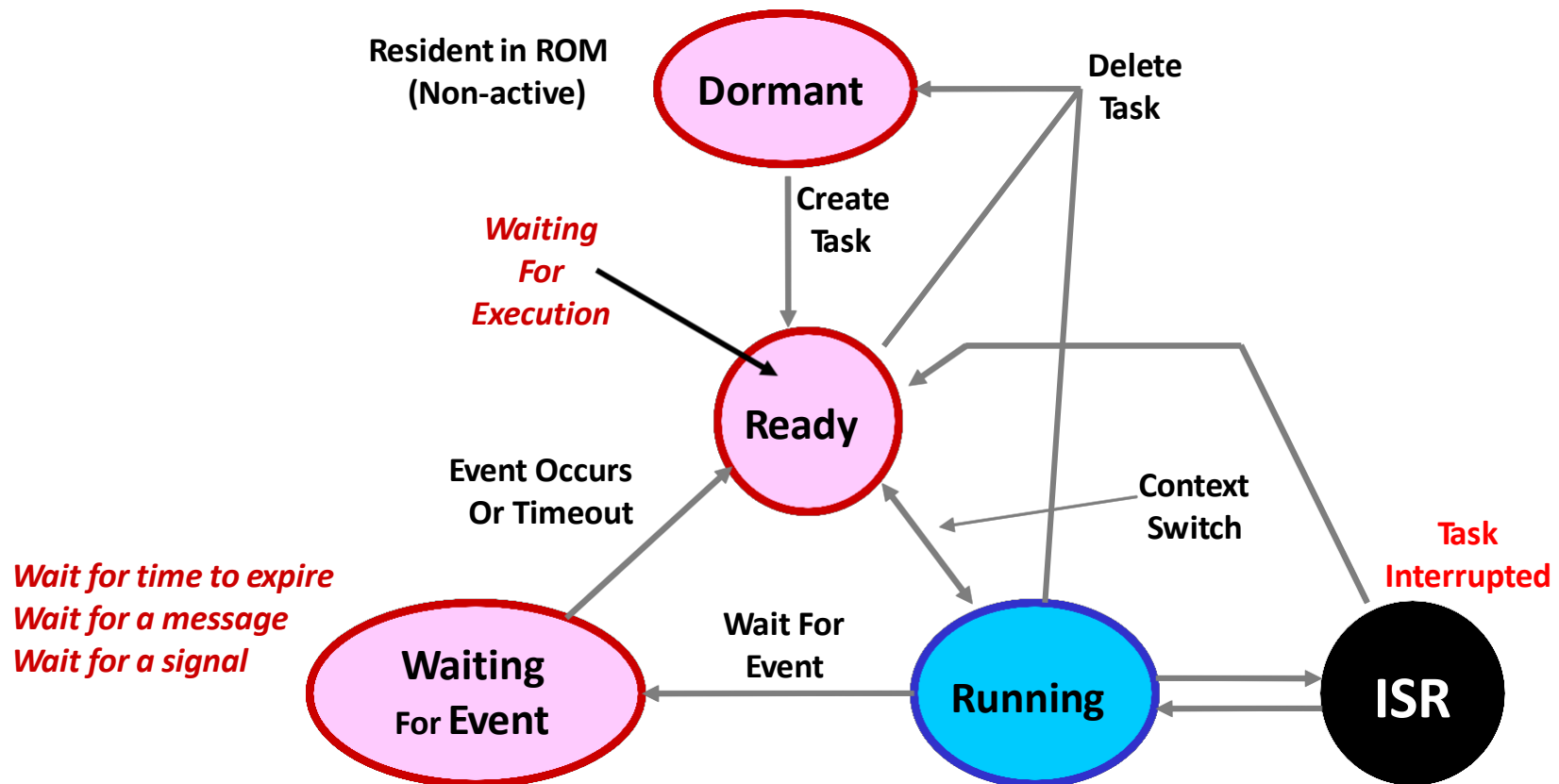
UCOSII

Gestión de Tareas

- Bucles infinitos (típicamente): en espera, listas (ready) o en ejecución
- UCOSII: menor número representa mayor prioridad.
- Idle (siempre) y Stat (si configurada) son las de menor prioridad (63 y 62).



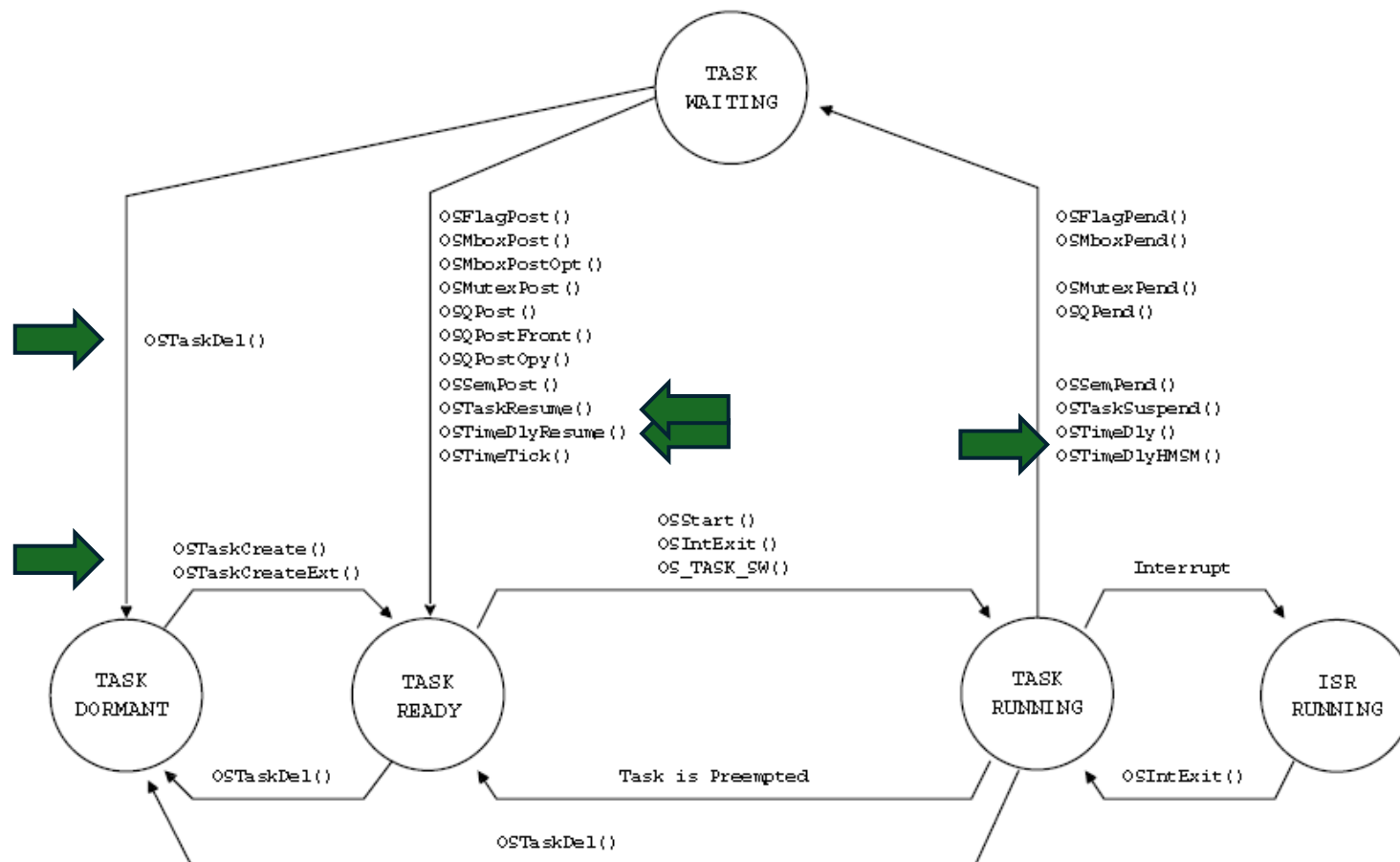
- Estados de tareas en UCOS.



UCOSII

Estados de Tasks

- Servicios que modifican el estado actual (algunos son eventos). Vamos a ver unos pocos por ahora. ➡



Estructura de una Tarea

```
void Task (void *pdata)
{
    .                /* Do something with 'pdata'                */
    for (;;) {        /* Task body, always an infinite loop.    */
        .
        .
        /* Must call one of the following services:            */
        /*  OSMboxPend()                                         */
        /*  OSFlagPend()                                         */
        /*  OSMutexPend()                                       */
        /*  OSQPend()                                           */
        /*  OSSemPend()                                          */
        /*  OSTimeDly()                                          */
        /*  OSTimeDlyHMSM()                                       */
        /*  OSTaskSuspend()   (Suspend self)                    */
        /*  OSTaskDel()       (Delete self)                     */
        .
        .
    }
}
```

Al ser un bucle infinito,
debe existir al menos una
llamada a estos servicios
del OS para cambiar al
estado Wait o Suspend

- Creación de tareas
 - OSTaskCreate()
 - OSTaskCreateExt()
- Eliminación de tareas
 - OSTaskDel()
 - OSTaskDelReq()

μC/OS-II Reference Manual

This chapter provides a reference to μC/OS-II services. Each of the user-accessible kernel services is presented in alphabetical order. The following information is provided for each of the services:

- A brief description
- The function prototype
- The filename of the source code
- The `#define` constant needed to enable the code for the service
- A description of the arguments passed to the function
- A description of the returned value(s)
- Specific notes and warnings on using the service
- One or two examples of how to use the function

UCOSII

TaskCreate

- Se debe pasar un puntero a la tarea, a sus argumentos, a la cima de la pila y la prioridad
- Se debe crear al menos una tarea antes de llamar a `OSStart()`.
- Como identificador de tarea se usa la prioridad. uCOSII solo acepta tareas con prioridades distintas.
- Una ISR no puede crear una tarea.

```
INT8U OSTaskCreate( void (*task)(void *p_arg),  
                    void *pdata,  
                    OS_STK *ptos,  
                    INT8U prio )
```

TaskCreate

OSTaskCreate()

```

INT8U OSTaskCreate(void    (*task)(void *pd),
                    void    *pdata,
                    OS_STK *ptos,
                    INT8U   prio);

```

<i>Chapter</i>	<i>File</i>	<i>Called from</i>	<i>Code enabled by</i>
4	OS_TASK.C	Task or startup code	OS_TASK_CREATE_EN

`OSTaskCreate()` creates a task so it can be managed by $\mu\text{C}/\text{OS-II}$. Tasks can be created either prior to the start of multitasking or by a running task. A task cannot be created by an ISR. A task must be written as an infinite loop, as shown below, and must not return.

`OSTaskCreate()` is used for backward compatibility with μ C/OS and when the added features of `OSTaskCreateExt()` are not needed.

Depending on how the stack frame is built, your task has interrupts either enabled or disabled. You need to check with the processor-specific code for details.

```
void Task (void *p_arg)
{
    .                               /* Do something with 'pdata'                */
    for (;;) {                      /* Task body, always an infinite loop. */
        .
        .
        /* Must call one of the following services:                */
        /*    OSMboxPend()                                           */
        /*    OSFlagPend()                                           */
        /*    OSMutexPend()                                          */
        /*    OSQPend()                                              */
        /*    OSSemPend()                                            */
        /*    OSTimeDly()                                            */
        /*    OSTimeDlyHMSM()                                         */
        /*    OSTaskSuspend()    (Suspend self)                     */
        /*    OSTaskDel()        (Delete self)                      */
        .
        .
    }
}
```

UCOSII

TaskCreate (argumentos)

- *task*: puntero al código de la tarea.
- *pdata*: puntero a un área de paso de parámetros de la tarea. p.e.: si la tarea gestiona un driver de puerto asíncrono, en la zona de memoria apuntada por *pdata* podemos tener la dirección, baudios, paridad, etc.
- *ptos*: puntero al top-of-stack de la tarea, posteriormente se describe la pila de una tarea.
- *prio*: .prioridad de la tarea, uCOSII da prioridad única y por tanto es un id de tarea.

Arguments

<code>task</code>	is a pointer to the task's code.
<code>pdata</code>	is a pointer to an optional data area used to pass parameters to the task when it is created. Where the task is concerned, it thinks it is invoked and passes the argument <code>pdata</code> . <code>pdata</code> can be used to pass arguments to the task created. For example, you can create a generic task that handles an asynchronous serial port. <code>pdata</code> can be used to pass this task information about the serial port it has to manage: the port address, the baud rate, the number of bits, the parity, and more.
<code>ptos</code>	is a pointer to the task's top-of-stack. The stack is used to store local variables, function parameters, return addresses, and CPU registers during an interrupt. The size of the stack is determined by the task's requirements and the anticipated interrupt nesting. Determining the size of the stack involves knowing how many bytes are required for storage of local variables for the task itself and all nested functions, as well as requirements for interrupts (accounting for nesting). If the configuration constant <code>OS_STK_GROWTH</code> is set to 1, the stack is assumed to grow downward (i.e., from high to low memory). <code>ptos</code> thus needs to point to the highest <i>valid</i> memory location on the stack. If <code>OS_STK_GROWTH</code> is set to 0, the stack is assumed to grow in the opposite direction (i.e., from low to high memory).
<code>prio</code>	is the task priority. A unique priority number must be assigned to each task, and the lower the number, the higher the priority (i.e., the task importance).

Returned Value

`OSTaskCreate()` returns one of the following error codes:

<code>OS_ERR_NONE</code>	if the function is successful.
<code>OS_ERR_PRIO_EXIST</code>	if the requested priority already exists.
<code>OS_ERR_PRIO_INVALID</code>	if <code>prio</code> is higher than <code>OS_LOWEST_PRIO</code> .
<code>OS_ERR_NO_MORE_TCB</code>	if μ C/OS-II doesn't have any more <code>OS_TCBs</code> to assign.
<code>OS_ERR_TASK_CREATE_ISR</code>	if you attempted to create the task from an ISR.

Notes/Warnings

1. The stack for the task must be declared with the `OS_STK` type.
2. A task must always invoke one of the services provided by μ C/OS-II to wait for time to expire, suspend the task, or wait for an event to occur (wait on a mailbox, queue, or semaphore). This allows other tasks to gain control of the CPU.
3. You should not use task priorities 0, 1, 2, 3, `OS_LOWEST_PRIO-3`, `OS_LOWEST_PRIO-2`, `OS_LOWEST_PRIO-1`, and `OS_LOWEST_PRIO` because they are reserved for use by μ C/OS-II.

UCOSII

TaskCreate

■ Ejemplo:

```
OSTaskCreate(Task1, NULL,  
&Task1Stk[TASK_STACK_SIZE - 1],  
TASK1_PRIOR);
```



```
#include <stdio.h>
#include "includes.h"    // Cabecera típica de µC/OS-II

#define TASK_STACK_SIZE  256
#define TASK1_PRIOR      5
#define TASK2_PRIOR      6

OS_STK Task1Stk[TASK_STACK_SIZE];
OS_STK Task2Stk[TASK_STACK_SIZE];

void Task1(void *pdata) {
    pdata = pdata; // Evitar warning
    while (1) {
        printf("Hello from Task 1\n");
        OSTimeDlyHMSM(0, 0, 1, 500); // 1.5 segundos
    }
}

void Task2(void *pdata) {
    pdata = pdata;
    while (1) {
        printf("Hello from Task 2\n");
        OSTimeDlyHMSM(0, 0, 1, 500); // 1.5 segundos
    }
}

int main(void) {
    OSInit(); // Inicializa el kernel

    // Crear las dos tareas
    OSTaskCreate(Task1, (void *)0, &Task1Stk[TASK_STACK_SIZE - 1], TASK1_PRIOR);
    OSTaskCreate(Task2, (void *)0, &Task2Stk[TASK_STACK_SIZE - 1], TASK2_PRIOR);

    OSStart(); // Arranca el sistema operativo
    return 0;
}
```

UCOSII

TaskCreate

- Ejemplo 2.
- Uso de parámetro al crear la tarea.
- ✓ Puede ser cualquier elemento externo de interés: Configuración de baudios, offset de un sensor...

```
#include <stdio.h>
#include "includes.h" // Cabecera típica de µC/OS-II
```

```
#define TASK_STACK_SIZE 256
#define TASK1_PRIO 5
#define TASK2_PRIO 6
```

```
OS_STK Task1Stk[TASK_STACK_SIZE];
OS_STK Task2Stk[TASK_STACK_SIZE];
```

```
void Task(void *pdata) {
    int id = *((int *)pdata); // Convertir el puntero a entero
    while (1) {
        printf("Hello from Task %d\n", id);
        OSTimeDlyHMSM(0, 0, 1, 500); // 1.5 segundos
    }
}
```

```
int main(void) {
    static int id1 = 1;
    static int id2 = 2;
```



solo visibles en este .c
dirección fija en memoria

```
OSInit(); // Inicializa el kernel
```

```
// Crear las dos tareas pasando el ID como segundo parámetro
OSTaskCreate(Task, (void *)&id1, &Task1Stk[TASK_STACK_SIZE - 1], TASK1_PRIO);
OSTaskCreate(Task, (void *)&id2, &Task2Stk[TASK_STACK_SIZE - 1], TASK2_PRIO);
```

```
OSStart(); // Arranca el sistema operativo
return 0;
```

```
}
```


TaskCreateExt

```

INT8U OSTaskCreateExt(void (*task)(void *pd),
                      void *pdata,
                      OS_STK *ptos,
                      INT8U prio,
                      INT16U id,
                      OS_STK *pbos,
                      INT32U stk_size,
                      void *pext,
                      INT16U opt);

```

opt contains task-specific options. The lower 8 bits are reserved by $\mu\text{C}/\text{OS-II}$, but you can use the upper 8 bits for application-specific options. Each option consists of one or more bits. The option is selected when the bit(s) is set. The current version of $\mu\text{C}/\text{OS-II}$ supports the following options:

`OS_TASK_OPT_SAVE_FP` specifies whether floating-point registers are saved. This option is only valid if your processor has floating-point hardware and the processor-specific code saves the floating-point registers.

<i>from</i>	<i>Code enabled by</i>
tup code	N/A

OS-II. This function serves the same purpose as additional information about your task to μ C/OS-II. It is created by a running task. A task cannot be created by the kernel, and must not return. Depending on how the task is disabled. You need to check with the processor vendor that the semantics are exactly the same as the ones for the older `OSTaskCreate()` function to this new and more powerful function. It is a replacement for the older `OSTaskCreate()` function instead of the older `OSTaskCreate()` function.

UCOSII

TaskCreateExt

```
OSTaskCreateExt(Task1,
                NULL,
                &Task1Stk[TASK_STACK_SIZE - 1], // Top-of-stack
                TASK1_PRIO,
                0,                                // ID = 0
                &Task1Stk[0],                    // Base del stack
                TASK_STACK_SIZE,
                NULL,                             // no datos extendidos
                0);                               // Sin opciones
```



```
#include <stdio.h>
#include "includes.h"
```

```
#define TASK_STACK_SIZE 256
#define TASK1_PRIO      5
#define TASK2_PRIO      6
```

```
OS_STK Task1Stk[TASK_STACK_SIZE];
OS_STK Task2Stk[TASK_STACK_SIZE];
```

```
void Task(void *pdata) {
    int id = *((int *)pdata);
    while (1) {
        printf("Hello from Task %d\n", id);
        OSTimeDlyHMSM(0, 0, 1, 500); // 1.5 segundos
    }
}
```

```
int main(void) {
    static int id1 = 1;
    static int id2 = 2;
```

```
    OSInit();
```

```
    // Crear tarea 1 con opciones extendidas
```

```
    OSTaskCreateExt(Task,
                    (void *)&id1,
                    &Task1Stk[TASK_STACK_SIZE - 1], // Tope del stack
                    TASK1_PRIO,
                    1,                                // ID de la tarea
                    &Task1Stk[0],                    // Base del stack
                    TASK_STACK_SIZE,
                    NULL,                             // Sin datos extendidos
                    OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR); // Opciones
```

```
    // Crear tarea 2 con opciones extendidas
```

```
    OSTaskCreateExt(Task,
                    (void *)&id2,
                    &Task2Stk[TASK_STACK_SIZE - 1],
                    TASK2_PRIO,
                    2,
                    &Task2Stk[0],
                    TASK_STACK_SIZE,
                    NULL,
                    OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
```

```
    OSStart();
    return 0;
```

UCOSII

TaskCreateExt

```
int main(void) {
    static TaskConfig cfg1 = {"Task1", 1500}; // 1.5 segundos
    static TaskConfig cfg2 = {"Task2", 1000}; // 1 segundo

    static DebugInfo dbg1 = {0};
    static DebugInfo dbg2 = {0};

    OSInit();

    // Crear tarea 1
    OSTaskCreateExt(Task,
        (void *)&cfg1,           // pdata: nombre y periodo
        &Task1Stk[TASK_STACK_SIZE - 1],
        TASK1_PRIO,
        1,
        &Task1Stk[0],
        TASK_STACK_SIZE,
        (void *)&dbg1,           // pext: contador ejecuciones
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    // Crear tarea 2
    OSTaskCreateExt(Task,
        (void *)&cfg2,
        &Task2Stk[TASK_STACK_SIZE - 1],
        TASK2_PRIO,
        2,
        &Task2Stk[0],
        TASK_STACK_SIZE,
        (void *)&dbg2,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    OSStart();
    return 0;
}
```

```
#include <stdio.h>
#include "includes.h"

#define TASK_STACK_SIZE 256
#define TASK1_PRIO 5
#define TASK2_PRIO 6

OS_STK Task1Stk[TASK_STACK_SIZE];
OS_STK Task2Stk[TASK_STACK_SIZE];

// Estructura para datos extendidos
typedef struct {
    char name[10];
    INT32U period_ms;
} TaskConfig;

// Función común para las tareas
void Task(void *pdata) {
    TaskConfig *cfg = (TaskConfig *)pdata; // Convertir puntero
    while (1) {
        printf("Hello from %s (period: %lu ms)\n", cfg->name, cfg->period_ms);
        OSTimeDlyHMSM(0, 0, 0, cfg->period_ms); // Delay según configuración
    }
}

int main(void) {
    static TaskConfig cfg1 = {"Task1", 1500}; // 1.5 segundos
    static TaskConfig cfg2 = {"Task2", 1000}; // 1 segundo

    OSInit();

    // Crear tarea 1 con todos los parámetros
    OSTaskCreateExt(Task,
        (void *)&cfg1,           // pdata
        &Task1Stk[TASK_STACK_SIZE - 1], // Tope del stack
        TASK1_PRIO,               // Prioridad
        1,                        // ID
        &Task1Stk[0],              // Base del stack
        TASK_STACK_SIZE,          // Tamaño del stack
        NULL,                      // Datos extendidos sin uso
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR); // Opciones

    // Crear tarea 2 con todos los parámetros
    OSTaskCreateExt(Task,
        (void *)&cfg2,
        &Task2Stk[TASK_STACK_SIZE - 1],
        TASK2_PRIO,
        2,
        &Task2Stk[0],
        TASK_STACK_SIZE,
        NULL,
        OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    OSStart();
    return 0;
}
```

UCOSII

TaskCreateExt

- Con chequeo de retorno de llamada a servicio

```
#include <stdio.h>
#include "includes.h"

#define TASK_STACK_SIZE 256
#define TASK1_PRIIO 5
#define TASK2_PRIIO 5
// ¡Misma prioridad para provocar error!

OS_STK Task1Stk[TASK_STACK_SIZE];
OS_STK Task2Stk[TASK_STACK_SIZE];

void Task(void *pdata) {
    char *name = (char *)pdata;
    while (1) {
        printf("Running %s\n", name);
        OSTimeDlyHMSM(0, 0, 1, 0);
    }
}
```

```
int main(void) {
    OSInit();

    INT8U err;

    // Crear primera tarea
    err = OSTaskCreateExt(Task,
                          (void *)"Task1",
                          &Task1Stk[TASK_STACK_SIZE - 1],
                          TASK1_PRIIO,
                          1,
                          &Task1Stk[0],
                          TASK_STACK_SIZE,
                          NULL,
                          OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    if (err == OS_NO_ERR) {
        printf("Task1 creada correctamente.\n");
    } else {
        printf("Error creando Task1: %d\n", err);
    }

    // Intentar crear segunda tarea con misma prioridad
    err = OSTaskCreateExt(Task,
                          (void *)"Task2",
                          &Task2Stk[TASK_STACK_SIZE - 1],
                          TASK2_PRIIO, // Igual que TASK1_PRIIO
                          2,
                          &Task2Stk[0],
                          TASK_STACK_SIZE,
                          NULL,
                          OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);

    if (err == OS_NO_ERR) {
        printf("Task2 creada correctamente.\n");
    } else if (err == OS_PRIIO_EXIST) {
        printf("Error: prioridad ya en uso (código %d).\n", err);
    } else {
        printf("Error creando Task2: %d\n", err);
    }

    OSStart();
    return 0;
}
```

UCOSII

TaskDel

- La tarea se pasa a estado “dormant”.
- El código no es borrado realmente, la tarea simplemente no es planificada por el kernel.
- Recibe como parámetro la prioridad de la tarea a borrar.
- Una ISR no puede borrar una tarea.
- No se puede borrar la tarea *idle*.

```
INT8U OSTaskDel( INT8U prio )
```

TaskDel

- Se puede pasar como parámetro el número de prioridad de la tarea a borrar.
- Se puede pasar OS_PRIO_SELF para eliminarse asimismo.
- Códigos devueltos:
 - OS_NO_ERROR
 - OS_TASK_DEL_IDLE intentas borrar tarea IDLE
 - OS_TASK_DEL_ERR no existe esa tarea
 - OS_PRIO_INVALID se intenta borrar una de mayor prioridad
 - OS_TASK_DEL_ISR intentas borrar una tarea desde ISR

TaskDelReq

- Solicitamos (Request) que queremos borrar una tarea.
- Se usa para no borrar directamente tareas de menor prioridad que están utilizando servicios o memoria compartida que puedan dejarnos “colgados”.
- Ejemplo siguiente:
 - La tarea 5 solicita borrar la tarea 10. *OSTaskDelReq(10)*
 - Hasta que devuelva **OS_TASK_NO_EXIST** la tarea 5 insiste en solicitar que la 10 se borre.
 - La tarea 10 se pregunta *OSTaskDelReq(OS_PRIO_SELF)* a sí misma, si la respuesta es **OS_TASK_DEL_REQ** significa que una tarea de mayor prioridad quiere que nos matemos (la 5).
 - Una vez liberados los recursos podemos matarnos con *OSTaskDel(OS_PRIO_SELF)*.

UCOSII

TaskDelReq

```
#include <stdio.h>
#include "includes.h"

#define TASK_STACK_SIZE 256
#define TASK5_PRIO      5
#define TASK10_PRIO     10

OS_STK Task5Stk[TASK_STACK_SIZE];
OS_STK Task10Stk[TASK_STACK_SIZE];

// Tarea 10: se autochequea y se elimina si hay petición
void Task10(void *pdata) {
    (void)pdata;
    while (1) {
        // Consultar si hay petición de borrado
        INT8U status = OSTaskDelReq(OS_PRIO_SELF);
        if (status == OS_TASK_DEL_REQ) {
            printf("Task10: Recibida petición de borrado. Liberando recursos...\n");
            // Aquí liberaríamos recursos (memoria, semáforos, etc.)
            OSTaskDel(OS_PRIO_SELF); // Autodestrucción
        }
        printf("Task10 ejecutando...\n");
        OSTimeDlyHMSM(0, 0, 1, 0); // Cada 1 segundo
    }
}

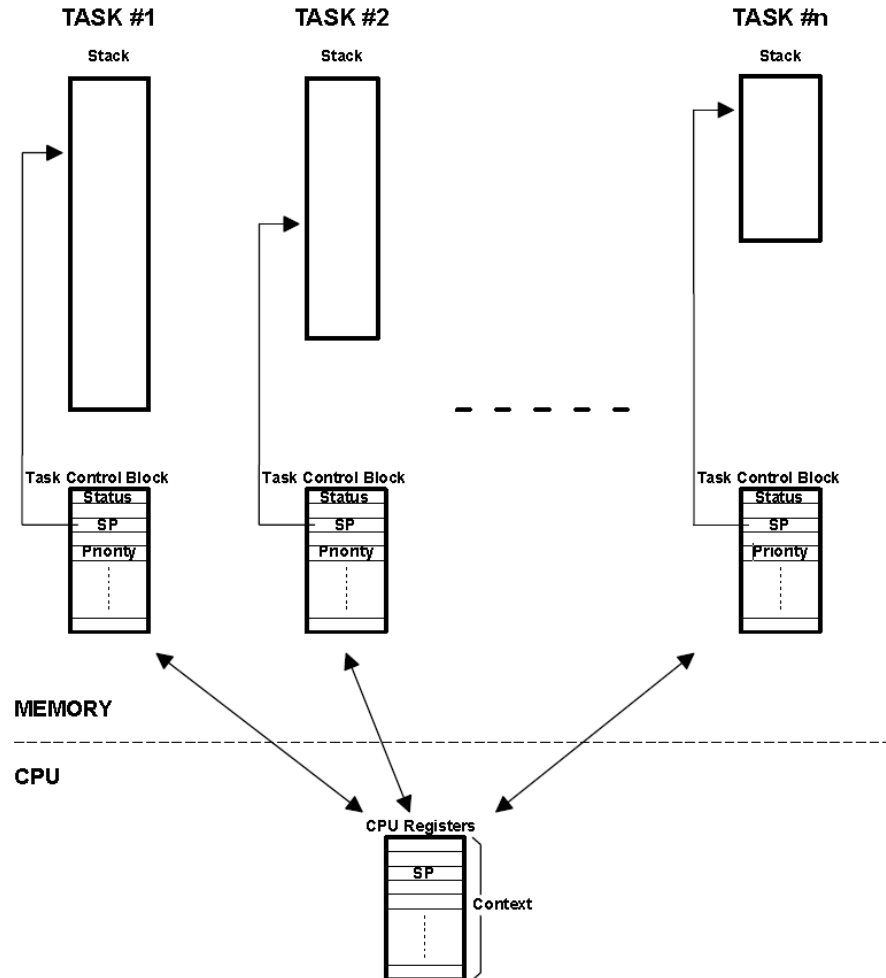
// Tarea 5: solicita borrar la tarea 10 hasta que no exista
void Task5(void *pdata) {
    (void)pdata;
    INT8U err;
    while (1) {
        err = OSTaskDelReq(TASK10_PRIO);
        if (err == OS_TASK_NO_EXIST) {
            printf("Task5: Tarea 10 ya no existe. Dejo de insistir.\n");
            break; // Salimos del bucle
        } else {
            printf("Task5: Solicitud de borrado enviada a tarea 10 (err=%d)\n", err);
        }
        OSTimeDlyHMSM(0, 0, 2, 0); // Cada 2 segundos
    }
    while (1) {
        printf("Task5 ejecutando...\n");
        OSTimeDlyHMSM(0, 0, 3, 0);
    }
}
```


Descriptores de la Tarea: Pila y TCB

- Recordad que cada tarea requiere una pila y un TCB: uso de memoria.
 - Cada tarea tiene su pila que se debe definir de tipo `OS_STK` y debe consistir en posiciones de memoria consecutivas.
 - Se le puede asignar espacio estáticamente en tiempo de compilación o dinámicamente en ejecución.
- Un TCB (*Task Control Block*) es una estructura de datos usada por el kernel para la administración de las tareas. Cuando una tarea es creada, el kernel le asigna un TCB propio.
- El TCB contiene:
 - La prioridad de la tarea
 - El estado de la tarea (Ready, Waiting ...)
 - Un puntero al stack de la tarea (TOS: Top-Of-Stack)
 - Otros datos relacionados con la tarea
- Los TCB ocupan un reducido espacio en memoria RAM. P. e. sólo 45 bytes para μ C/OS-II con CPU 80x86

UCOSII

Descriptores de la Tarea: Pila y TCB



Suspensión y recuperación de tareas

■ OSTaskSuspend(INT8U prio)

- **Función:** Suspende la tarea indicada por su prioridad.
- **Efecto:** La tarea deja de estar lista para ejecutarse y pasa al estado **SUSPENDIDO**.
- **Parámetro:**
 - prio: Prioridad de la tarea a suspender.
- **Notas:**
 - Si se suspende la tarea actual (OS_PRIO_SELF), el planificador seleccionará otra tarea lista.
 - No se puede suspender la tarea **Idle** (prioridad más baja).

•Estos servicios son útiles para **controlar tareas dinámicamente**, por ejemplo: Pausar tareas que no son críticas temporalmente.

•Reactivar tareas cuando se cumplen ciertas condiciones.

Suspensión y recuperación de tareas

- **OSTaskResume(INT8U prio)**
 - **Función:** Reactiva (resume) la tarea previamente suspendida.
 - **Efecto:** La tarea vuelve al estado **READY** si no hay otras condiciones que la bloqueen.
 - **Parámetro:**
 - prio: Prioridad de la tarea a reactivar.
 - **Notas:**
 - Solo funciona si la tarea estaba suspendida.
 - Si la tarea estaba bloqueada por otro recurso, seguirá bloqueada.

❖ Ejercita con AI (ChatGPT, Copilot, Gemini...)

Gestión de Tiempo

■ Interrupción de Reloj (Tick)

- μ C/OS-II requiere una interrupción periódica (ISR), que puede ser generada por un timer de hardware (es típico de 10 a 1000 ticks/seg).
- Es llamada '*Clock Tick*' o '*System Tick*', a mayor frecuencia genera mayor *overhead* pero permite mayor resolución.
- Forma parte de la implementación de μ C/OS-II para NIOS II
- La ISR invoca un servicio del RTOS para procesar el 'tick', que:
 - Permite que una tarea **suspenda** su ejecución un cierto lapso de tiempo
 - Proporciona **timeouts** a otros servicios del RTOS (flags, semáforos, mensajes, etc), para impedir que ciertas tareas queden bloqueadas eternamente a la espera de eventos que no ocurren, o sincronizaciones fallidas, y de ese modo evitar bloqueos (deadlocks)
- Los procesadores (Ej: ARM/Cortex) incluyen un SysTick Timer como parte innata de la CPU.
- NIOS II tiene Interval_Timer como Timer para uCOS.

Gestión de Tiempo

- Servicios de Gestión de tiempo (Suspensión)
 - μ C/OS-II permite a una tarea suspender su ejecución por cierto tiempo
 - Un cierto número de 'ticks': `OSTimeDly(ticks)`
 - Un cierto tiempo : `OSTimeDlyHMSM(hr, min, sec, ms)`
 - Esta acción siempre genera un cambio de contexto, porque la tarea queda en estado de espera (WAITING).
 - Suponiendo un clock tick de 100 Hz (10 ms), una tarea que revise un teclado cada 100ms podría tener el siguiente esquema:

```
void Keyboard_Scan_Task (void *p_arg)
{
    for (;;) {
        OSTimeDly(10);          /* Cada 100 mS */
        Scan keyboard;
    }
}
```

Gestión de Tiempo

■ Servicios de Gestión de tiempo (Suspensión)

void OSTimeDly(INT16U ticks)

- Permite a una tarea esperar entre 0 y 65535 ticks de reloj. La resolución es de 1 tick.
- Se produce un cambio de contexto que planifica otra tarea.
- La tarea que llama a **OSTimeDly()** se pone “lista” para ejecución cuando el tiempo especificado expira o si otra tarea cancela el retardo llamando a **OSTimeDlyResume()**.

INT8U OSTimeDlyHMSM (INT8U hours, min, sec, mil)

- Permite especificar el retardo en unidades de tiempo.
- La resolución se define en **OS_CFG.H**, **OS_TICKS_PER_SEC**.

Gestión de Tiempo

■ Servicios de Gestión de tiempo (Suspensión)

`void OSTimeDlyResume( INT8U prio );`

- Permite a una tarea cancelar el retardo ejecutado por otra tarea.

`INT32U OSTimeGet( void );`

- Permite consultar el valor del reloj del sistema. Es un contador de 32 bits inicializado al llamar a `OSStart()` e incrementado con la interrupción de reloj.

`void OSTimeSet( INT32U );`

- Permite ajustar el valor del reloj del sistema.

Gestión de Tiempo

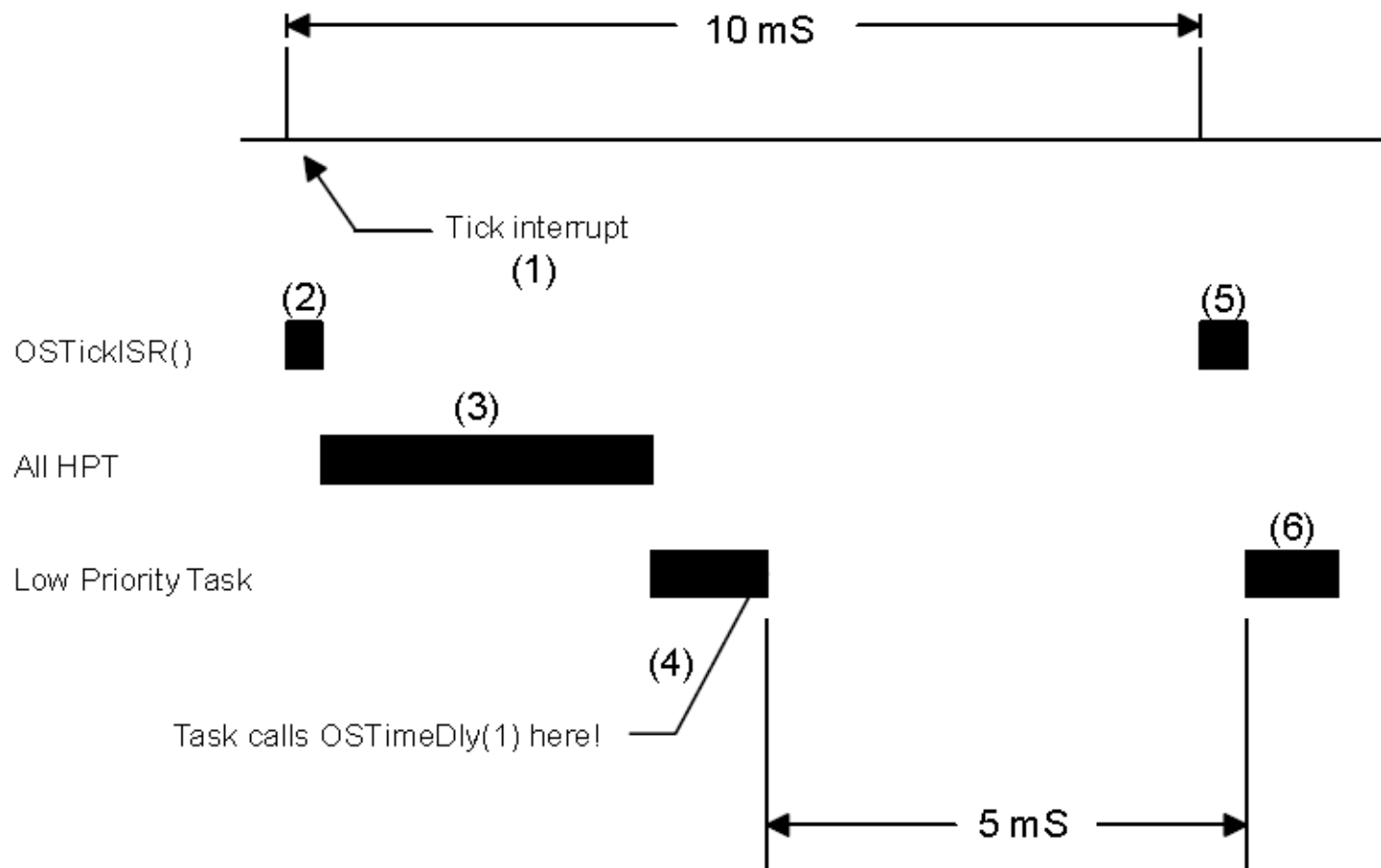
- Servicios de Gestión de tiempo (Timeout)
- Los servicios que quedan en suspenso a la espera de eventos permiten definir timeouts, para no quedar esperando por siempre y evitar bloqueos
- Por ejemplo, un conversor deja de generar su EOC en el tiempo

```
void ADCTask (void *p_arg)
{
    void *p_msg; OS_ERR err;
    for (;;) {
        Start ADC;
        p_msg = OSMboxPend(..., ..., 10, &err);
        if (err == OS_NO_ERR) {
            Read ADC and Scale; ... ; ... ; ... ;
        } else { /* Problema con el ADC! */ }
    }
}
```

UCOSII

Gestión de Tiempo

- Resolución
 - Puede ser menor que un tick



Interrupciones

- QUITAN el control de la CPU a las tareas convencionales, con lo que pueden afectar a la previsibilidad temporal.
- En general son siempre aceptadas aunque pueden ser bloqueadas desde el RTOS.
- Una rutina de servicio de interrupción debe ser lo más breve posible:
 - Aceptar la interrupción y leer o escribir en el recurso
 - Enviar una señal a una tarea convencional informando de la novedad
 - Si se requiere mucha velocidad una ISR puede escribirse en ensamblador. En caso contrario en ANSI C.

Interrupciones

■ Estructura de una ISR

el HAL casi todo

```
Save all CPU registers;
Call OSIntEnter() or increment OSIntNesting;
if (OSIntNesting ==1) {
    OSTCBCur->OSTBStkPtr = SP;
}
Clear interrupting device;
Re-enable interrupts (optional);
Execute user code to service ISR;
Call OsIntExit();
Restore all CPU registers;
Execute a return from interrupt instruction;
```

Interrupciones

■ Usando el HAL

```
int alt_irq_register (alt_u32 id, void* context,  
                     void (*isr)(void*, alt_u32))
```

•alt_u32 id

- Número de IRQ (interrupt request) que se quiere asociar.
- Valor típico: Depende del dispositivo; por ejemplo, 0 para el primer periférico, 1 para el siguiente, etc.
- Cada periférico tiene asignado un ID en el sistema SOPC/Qsys.

•void* context

- Puntero a datos que se pasarán al manejador cuando se invoque.
- Uso: Puede ser NULL si no se necesita contexto adicional.
- Ejemplo: Dirección de una estructura con información del dispositivo.

•alt_isr_func handler

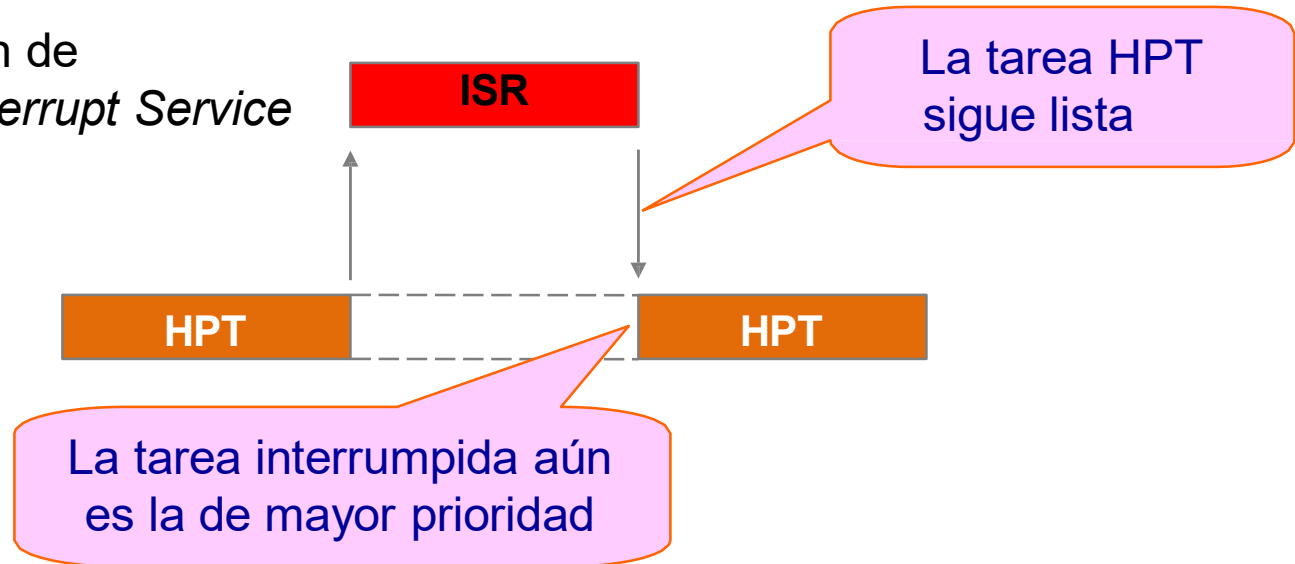
- Función que actuará como **ISR (Interrupt Service Routine)**.

Interrupciones

■ Cambio de contexto

Rutina de atención de interrupciones: *Interrupt Service Routine (ISR)*

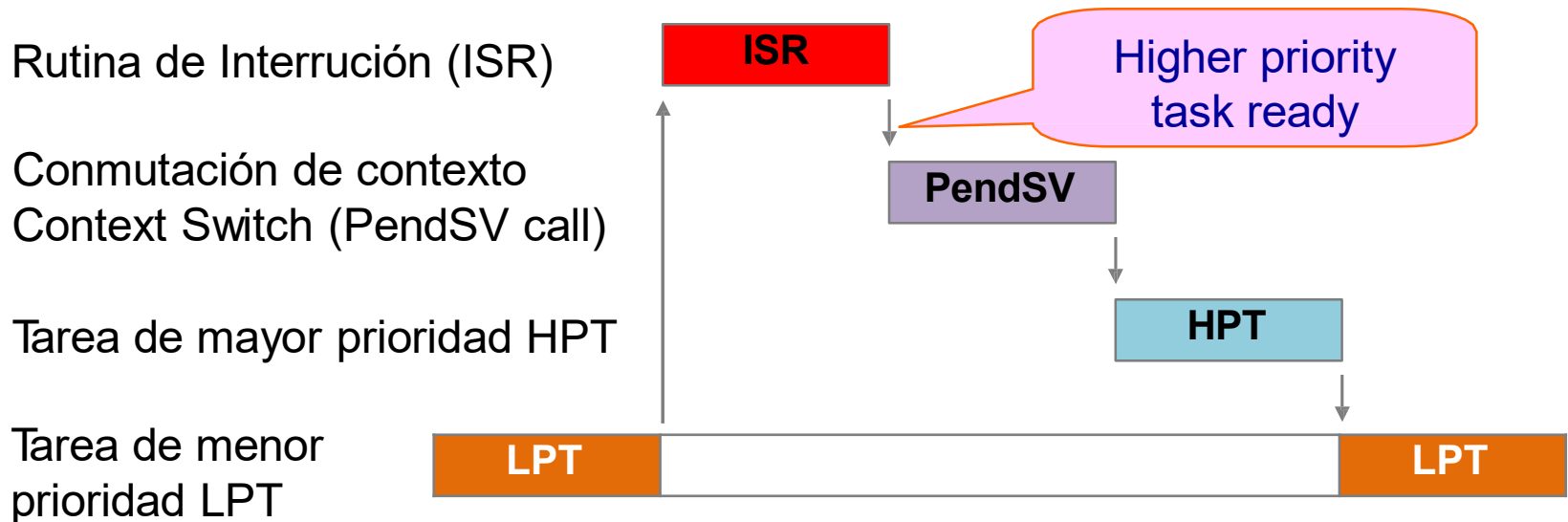
Tarea de mayor prioridad: *HPT = High Priority Task*



Cuando la tarea interrumpida es la de mayor prioridad al finalizar la ISR, se retoma la misma tarea

Interrupciones

■ Cambio de contexto



Cuando la tarea interrumpida es de menor prioridad y existe una de mayor prioridad lista para ejecución se conmuta de contexto

Interrupciones

■ Ejemplo

```
#include "key_codes.h" // defines values for KEY1, KEY2, KEY3
#include "system.h"
#include "sys/alt_irq.h"
#include <stdio.h>
#include "includes.h"

/* Definition of Task Stacks */
#define TASK_STACKSIZE 2048
OS_STK TimeShow_task_stk[TASK_STACKSIZE];
OS_STK interval_timer_task_stk[TASK_STACKSIZE];
/* Definition of Task Priorities */
#define interval_timer_TASK_PRIORITY 2
#define TimeShow_TASK_PRIORITY 3

void interval_timer_task(void* );
void pushbutton_ISR( );
static int Dec2Bcd(int);
void Update_HEX_display(int);
void TimeShow_task(void* );
void LCD_cursor( int, int );
void LCD_text( char * );
void LCD_cursor_off( void );

volatile int key_pressed = KEY2;
volatile int horas = 12;
volatile int minutos = 0;
volatile int segundos = 0;
volatile int milisegundos = 0;
```


Interrupciones

```
int main(void)
{
    volatile int * KEY_ptr = (int *) PUSHBUTTONS_BASE;
    char text_top_row[40] = "Reloj Digital\0";
    char text_bottom_row[40] = "Ricardo J. Colom\0";

    /* output text message to the LCD */
    LCD_cursor (0,0); // set LCD cursor location to top row
    LCD_text (text_top_row);
    LCD_cursor (0,1); // set LCD cursor location to bottom row
    LCD_text (text_bottom_row);
    LCD_cursor_off (); // turn off the LCD cursor

    /* set 3 mask bits to 1 (bit 0 is Nios II reset) */
    *(KEY_ptr + 2) = 0xE;
    *(KEY_ptr + 3) = 0;

    alt_irq_register(PUSHBUTTONS_IRQ, NULL, pushbutton_ISR);

    OSInit();
    OSTaskCreateExt(interval_timer_task, ... ..
    //RESTO DE CREACION DE TAREAS
    OSStart();
    return 0;
}
```

```

#include "key_codes.h" // defines values for KEY1, KEY2, KEY3
#include "system.h"
#include "sys/alt_irq.h"
#include "includes.h"
extern volatile int key_pressed;
extern volatile int horas;
extern volatile int minutos;
extern volatile int segundos;
/*****
 * Pushbutton - Interrupt Service Routine
 *
 * This routine checks which KEY has been pressed. If it is KEY1 or KEY2, it writes this
 * value to the global variable key_pressed. If it is KEY3 then it loads the SW switch
 * values and stores in the variable pattern
 *****/
void pushbutton_ISR( )
{
    OSIntEnter();
    volatile int * KEY_ptr = (int *) PUSHBUTTONS_BASE;
    int press;
    press = *(KEY_ptr + 3); // read the pushbutton interrupt register
    *(KEY_ptr + 3) = 0; // Clear the interrupt
    if (press & 0x2) // KEY1
    {
        key_pressed = KEY1;
        segundos = 0;
        // CODIGO NECESARIO PARA ATENDER LA ISR..
    }
    else
    {
        horas++;
    }
    OSIntExit();
}

```